Qwen 模型结构详解

Qwen（通义千问）是阿里巴巴的开源大语言模型系列。它的核心架构与 GPT、LLaMA 等现代大语言模型一样，主要采用 Decoder-only Transformer（仅解码器 Transformer）。

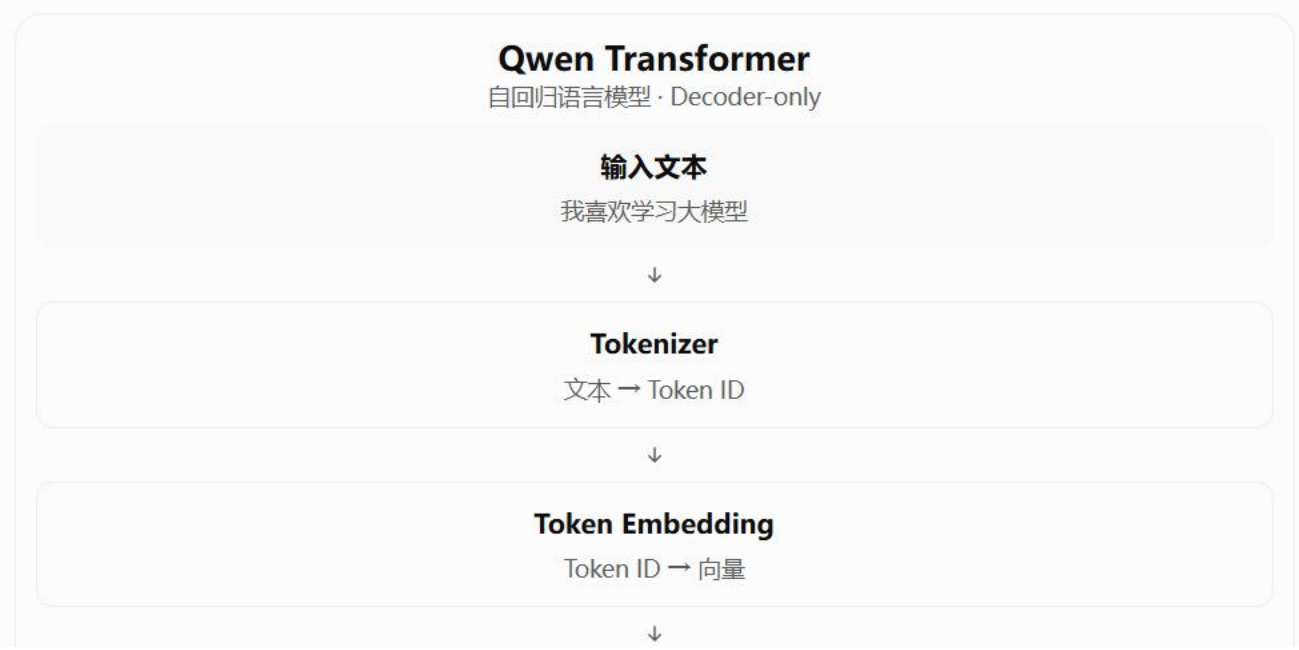
先说结论：

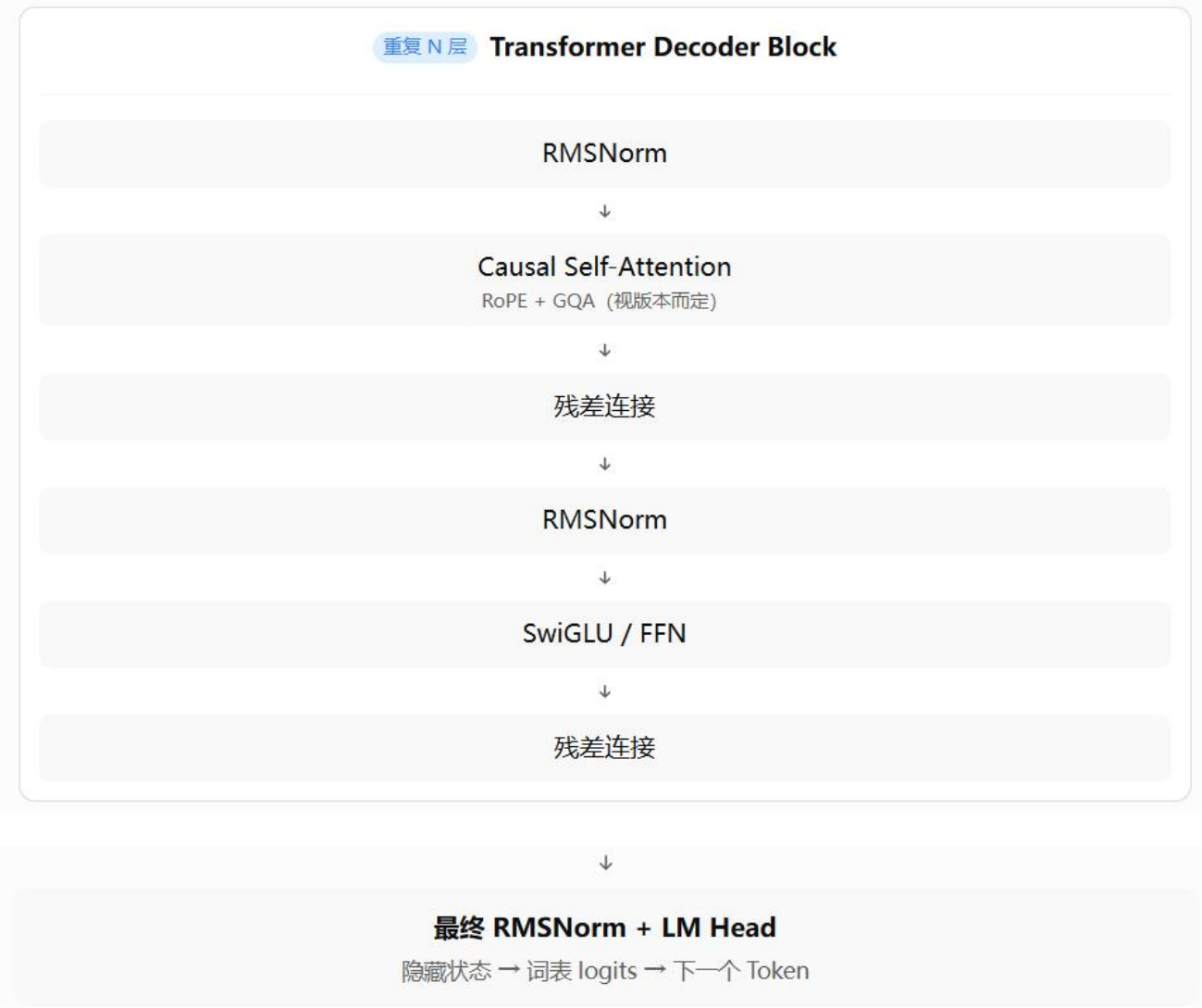
> Qwen 不是完全不同于 Transformer 的新架构，而是在传统 Transformer 解码器基础上，对位置编码、注意力机制、归一化、激活函数、词表和训练方式等进行优化。

不过，Qwen 不同版本之间的结构差异很大。Qwen2、Qwen2.5、Qwen3，以及 Qwen3-MoE 不能完全当成同一个架构。

一、先看整体结构

以常见的 Qwen2 / Qwen2.5 类 Decoder-only 模型为例：





这就是它与传统 GPT 类 Transformer 最重要的共同点：一堆相同的 Decoder Block 叠加起来，最后预测下一个 token。

二、Qwen 与传统 Transformer 到底有什么区别？

这里的“传统 Transformer”需要区分两种：

- 原始 Transformer（2017）：Encoder + Decoder，使用 Multi-Head Attention、LayerNorm、绝对位置编码等。
- 早期 GPT 类模型：Decoder-only Transformer，但通常采用标准多头注意力、LayerNorm、GELU 等组件。

Qwen2.5 的官方模型卡明确列出：RoPE、SwiGLU、RMSNorm、Attention QKV bias，并使用 GQA。

组件	原始/早期 Transformer 常见做法	Qwen2.5 类模型
模型结构	Encoder-Decoder 或 Decoder-only	Decoder-only
注意力	MHA	GQA

位置编码	绝对位置编码	RoPE
归一化	LayerNorm	RMSNorm
FFN 激活	ReLU / GELU	SwiGLU
注意力 Bias	视实现而定	QKV bias
训练方式	任务相关	大规模预训练 + 后训练

重点：这些改动并非 Qwen 独创。 很多现代 LLM，包括 LLaMA 系列，也采用类似的结构。Qwen 的特色在于具体架构配置、Tokenizer、训练数据、训练方法以及不同版本的工程优化。

三、最重要的特殊点：GQA 分组查询注意力

这是理解 Qwen2.5 架构时最值得掌握的一个地方。

1. 传统 MHA

标准 Multi-Head Attention（多头注意力）中，每个 Query Head 都有自己对应的 Key 和 Value Head。

例如：

```
Q1 → K1, V1
Q2 → K2, V2
Q3 → K3, V3
Q4 → K4, V4
Q5 → K5, V5
Q6 → K6, V6
Q7 → K7, V7
Q8 → K8, V8
```

8 个 Query Head，8 个 KV Head。

2. Qwen2.5 的 GQA

Grouped Query Attention 将多个 Query Head 分成一组，共享一个 Key/Value Head。

例如 8 个 Query Head、2 个 KV Head：

```
Q1 ─┐
Q2 ─┤
Q3 ─┼── K1, V1
Q4 ─┘

Q5 ─┐
Q6 ─┤
Q7 ─┼── K2, V2
Q8 ─┘
```

Qwen2.5-7B 使用 28 个 Query Heads 和 4 个 KV Heads；Qwen2.5-72B 使用 64 个 Query Heads 和 8 个 KV Heads。

3. 为什么 GQA 更好？

在自回归生成时，模型需要缓存之前所有 Token 的 K 和 V，这就是 KV Cache。

传统 MHA：

$$KV\ \ Cache \propto 2 \times L \times H \times d$$

GQA：

$$KV\ \ Cache \propto 2 \times L \times H_{KV} \times d$$

其中：

L：序列长度

H：Query Head 数量

H_{KV} ：KV Head 数量

d：每个 Head 的维度

假设 Query Heads = 32，KV Heads = 8，那么：

$$8/32=25\%$$

KV Cache 大约只有标准 MHA 的四分之一。

实际意义：

长上下文推理时，显存占用更低。

自回归生成时，KV Cache 读取量减少。

同等显存下，可以支持更大的 Batch 或更长上下文。

但注意：GQA 并不是把整个模型参数量降低到四分之一，因为 Q 投影、FFN 等部分仍然存在。

四、RoPE：Qwen 的旋转位置编码

1. 传统 Transformer 的位置编码

原始 Transformer 使用正弦、余弦位置编码：

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d}}\right)$$

$$PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

然后把位置编码加到 Token Embedding 上：

X=TokenEmbedding+PositionEmbedding

2. Qwen 的 RoPE

Qwen2.5 使用 Rotary Positional Embeddings（旋转位置编码）。它不是简单把位置向量加到输入，而是对 Attention 中的 Query 和 Key 做旋转变换。

可以简单理解为：

```
Token Embedding
      |
      ▼
   Q / K
      |
      ▼
  RoPE 旋转
      |
      ▼
Attention(Q, K, V)
```

二维旋转可以写成：

$$\begin{bmatrix} x_1' \\ x_2' \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

其中：

θ=pos· ωi

3. RoPE 的优势

RoPE 的核心优势是让 Attention 分数能够体现相对位置信息。

这对于：

长文本建模

代码理解

文档问答

多轮上下文

都很重要。

不过，RoPE 本身不等于无限上下文。上下文长度还取决于训练长度、位置频率配置和推理方法。

例如 Qwen2.5-7B 的模型卡给出 131,072 token 的上下文长度，而一些 0.5B / 1.5B 模型配置则不同。

五、RMSNorm：比传统 LayerNorm 更简洁的归一化

1. 传统 LayerNorm

LayerNorm 会计算均值和方差：

$$\mu = \frac{1}{d}\sum_{i=1}^{d} x_i$$

$$\sigma^2 = \frac{1}{d}\sum_{i=1}^{d}(x_i - \mu)^2$$

$$LN(x) = \gamma \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta$$

2. RMSNorm

RMSNorm 只计算均方根：

$$RMS(x) = \sqrt{\frac{1}{d}\sum_{i=1}^{d} x_i^2 + \epsilon}$$

$$RMSNorm(x) = \gamma \frac{x}{RMS(x)}$$

Qwen2.5 使用 RMSNorm，并采用 Pre-Norm 结构。

3. 为什么使用 RMSNorm?

主要原因：

计算更简单。

不需要显式计算均值。

适合大规模 Transformer 训练。

配合 Pre-Norm 有助于训练稳定性。

注意：RMSNorm 并不保证一定比 LayerNorm 效果好，它是现代大模型常见的架构选择。

六、SwiGLU：Qwen 的前馈网络

传统 Transformer 的 FFN：

$$FFN(x) = W_2\sigma(W_1 x)$$

其中 σ 可能是 ReLU 或 GELU。

Qwen2.5 使用 SwiGLU。

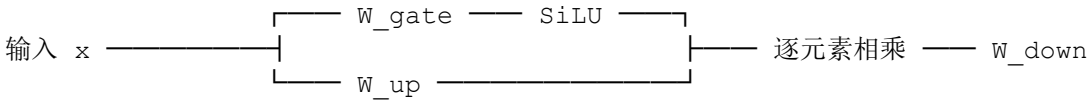
典型形式：

$$SwiGLU(x) = W_{down}\left[\mathrm{SiLU}(xW_{gate}) \odot (xW_{up})\right]$$

其中：

$SiLU(x)=x\sigma(x)$

结构可以理解为：



与普通 FFN 相比，SwiGLU 多了一个门控分支，允许模型学习哪些特征应该被增强或抑制。

参数量上的影响

普通 FFN：

$2dd_{ff}$

SwiGLU：

$3dd_{ff}$

因为多了一个 Gate 投影。

但实际模型通常会调整中间维度，使总参数量保持在合理范围内。

七、Qwen 的 Attention 具体怎么计算？

以 Qwen2.5 为例，单层 Attention 的流程：

Step 1：输入隐藏状态

$X\in R^{B\times L\times d}$

B：Batch Size

L：序列长度

d：隐藏维度

Step 2：生成 Q、K、V

$Q=XW_Q$
$K=XW_K$
$V=XW_V$

Qwen2.5 保留 Attention QKV bias。

Step 3：RoPE

对 Q、K 应用旋转位置编码：

$Q' = \text{RoPE}(Q)$
$K' = \text{RoPE}(K)$

Step 4：GQA

将 Query Heads 分组，与较少的 KV Heads 共享。

Step 5：因果 Mask

Qwen 是自回归模型，当前位置不能看到未来 Token：

$$M_{ij} = \begin{cases} 0 & j \le i \\ -\infty & j > i \end{cases}$$

Step 6：计算 Attention

$$Attention(Q,K,V) = softmax\left(\frac{QK^T}{\sqrt{d_k}} + M\right)V$$

Step 7：输出投影

$O = Attention(Q,K,V)W_o$

八、Qwen2.5-7B 的真实架构参数

下面用一个具体模型来理解，不只是抽象公式。

Qwen2.5-7B

典型开源 Dense Decoder-only 模型

参数量

7.61B

Transformer 层数

28

Query Heads

28

KV Heads

4

上下文长度

131,072

隐藏维度

3,584

参数和层数来自 Qwen2.5-7B 官方模型卡；隐藏维度为该模型常用配置。

关键配置

```
config = {
    "hidden_size": 3584,
    "num_hidden_layers": 28,
    "num_attention_heads": 28,
    "num_key_value_heads": 4,
    "hidden_act": "silu",
    "norm_type": "RMSNorm",
    "position_embedding": "RoPE",
    "attention_bias": True,
}
```

这里的 `config` 是帮助理解的简化示意，不是完整官方配置文件。

九、Qwen 和 LLaMA 的区别

如果你学习大模型源码，Qwen 和 LLaMA 的结构其实非常接近。

方面	Qwen2.5	LLaMA 3 类模型
主体	Decoder-only Transformer	Decoder-only Transformer
RoPE	是	是

RMSNorm	是	是
SwiGLU	是	是
GQA	是	是
QKV Bias	Qwen2.5 使用	LLaMA 3 常见实现不使用
Tokenizer	Qwen 自有词表	LLaMA 自有词表
训练数据	Qwen 训练数据	Meta 训练数据
具体层数/维度	不同	不同

因此，Qwen 的能力差异不应该简单归因于“它用了不同于 Transformer 的结构”。

真正的差异来自：

模型规模与宽深配置。

Tokenizer 和词表设计。

预训练数据质量与数量。

SFT、偏好优化、RL 等后训练。

推理优化和长上下文技术。

十、Qwen2.5、Qwen3、Qwen-MoE 要区分

你问的是“Qwen 结构”，这里非常重要：Qwen 是一个模型家族，不是一个固定架构。

Qwen2 / Qwen2.5

Dense 为主

典型结构是 Decoder-only Transformer + GQA + RoPE + RMSNorm + SwiGLU。

适合学习现代大模型基本架构。

Qwen3

新一代模型系列

Qwen3 仍然属于 Transformer 家族，但部分模型可能采用 MoE 等不同设计。不能把 Qwen2.5 的全部参数直接套用到 Qwen3。

Qwen-MoE

混合专家模型

在 Transformer 的 FFN 部分引入多个专家网络，每个 Token 只激活其中一部分专家。

重点是稀疏计算，不是把所有参数都同时计算。

Dense 和 MoE 的区别

Dense：

```
Token
  ↓
Attention
  ↓
FFN（所有参数参与计算）
  ↓
Output
```

MoE：

```
Token
  ↓
Attention
  ↓
Router
  ↓
选择 Top-K Experts
  ├── Expert 1
  ├── Expert 2
  └── Expert 3
  ↓
加权合并
  ↓
Output
```

这就是为什么一些 MoE 模型可以拥有非常大的总参数量，但每个 Token 的实际激活参数量相对较小。

十一、Qwen 的长上下文特点

Qwen2.5 系列支持长上下文。官方 Qwen2.5-1M 技术报告介绍了：

GQA，用于高效 KV Cache。

RoPE，编码位置信息。

长上下文训练。

长度外推方法。

稀疏注意力等推理优化。

但要注意：

普通 Qwen2.5-7B 和 Qwen2.5-1M 不是同一个上下文配置。

普通模型的上下文长度、训练长度和 1M 长上下文模型的技术不能混为一谈。

十二、如果你要学习 Qwen 源码，应该看哪些文件？

如果你准备使用 Python / PyTorch 理解 Qwen，可以重点看 Hugging Face Transformers 中的实现：

Qwen2 源码学习路线

1. `Qwen2Model`

整个 Transformer 主干，Embedding、Decoder Layers、最终 Norm。

2. `Qwen2DecoderLayer`

理解 Attention、残差连接、FFN 的组合。

3. `Qwen2Attention`

重点学习 QKV 投影、GQA、RoPE、KV Cache。

4. `Qwen2MLP`

重点学习 Gate、Up、Down 三个线性层以及 SwiGLU。

5. `Qwen2RMSNorm`

理解 RMSNorm 的数学公式和实现。

6. `Qwen2Tokenizer / Qwen2Config`

理解 Tokenizer、模型维度和配置。

官方源码可以从 Hugging Face Transformers 中查看，搜索 `modeling_qwen2.py`。

最后总结：Qwen 的核心特典

如果你只记住下面几条：

Qwen 是 Transformer，不是完全不同的模型结构。

Qwen2.5 使用 Decoder-only Transformer。

GQA：减少 KV Cache，降低长上下文推理成本。

RoPE：更适合现代自回归 Transformer 的位置信息建模。

RMSNorm + Pre-Norm：简化归一化，并适合大规模训练。

SwiGLU：门控前馈网络，提升非线性表达能力。

Qwen2.5 使用 Attention QKV bias，这是与很多 LLaMA 类实现的一个具体差异。

Qwen 的实际能力不仅来自架构，也来自训练数据、Tokenizer、后训练和工程优化。

推荐下一步

如果你想真正理解 Qwen 的底层结构，建议按照这个顺序学习：

Transformer 的 Multi-Head Attention。
GQA 与 KV Cache 的计算。
RoPE 数学公式与 PyTorch 实现。
SwiGLU、RMSNorm。
完整 Qwen2DecoderLayer 源码。